The mavericks

# A3-PS023-TC246

## Minimal Two-Node Cloud Hypervisor VM Controller

AIORI Hackathon 3

# The Mavericks


01 Problem Description

02 Background

03 Literature & RFC Review

04 Proposed Solution

05 Optimization Proposed

06 Solution Architecture & Design

07 Data Flow Diagram

08 Deployment Timeline

09 Bibliography

The Mavericks

# Problem Description

Managing virtual machines across multiple compute hosts requires reliable coordination, efficient resource allocation, and secure communication. Without a centralized management mechanism, operations such as creating, starting, stopping, listing, and deleting virtual machines can become difficult to coordinate, especially when multiple requests compete for limited system resources. This creates challenges in maintaining consistent VM states, preventing resource overcommitment, and ensuring reliable operation across hosts.

1. Develop a lightweight controller to manage VM lifecycle operations across two KVM-capable Linux hosts using Cloud Hypervisor.
2. Implement host selection and resource reservation mechanisms to prevent memory overcommitment and ensure safe VM placement.
3. Establish secure communication between the controller and worker agents, with continuous health monitoring and status tracking.
4. Ensure consistent VM state, recover from worker failures, and perform safe resource cleanup for reliable multi-host VM management.

The Mavericks

# Background

## VIRTUALIZATION AND RESOURCE MANAGEMENT

Virtualization enables multiple virtual machines (VMs) to run on shared physical hardware, improving infrastructure utilization and flexibility. In cloud environments, efficient VM placement and resource provisioning are important for balancing resource availability and application requirements. Calheiros et al. [1] introduced CloudSim, a simulation toolkit for evaluating cloud computing environments and resource provisioning algorithms, highlighting the importance of systematic resource management and performance evaluation [1].

## CHALLENGES IN MULTI-HOST VM MANAGEMENT

Managing VMs across multiple hosts introduces challenges involving resource contention, VM placement, and maintaining service quality under changing workloads. Beloglazov et al. [2] examined resource allocation and scheduling strategies for cloud data centres, emphasizing the role of resource-aware allocation policies in improving efficiency while meeting quality-of-service requirements [2]. These findings motivate careful memory allocation, worker selection, and resource tracking in a multi-host VM controller.

## PROPOSED OPTIMIZATION

Building on these research directions, the PS-023 project proposes a lightweight centralized controller for managing virtual machines across two KVM-capable Linux hosts using Cloud Hypervisor. The proposed system combines memory-aware scheduling, atomic resource reservations, worker health monitoring, and VM state reconciliation. A central controller coordinates requests while worker agents execute host-level operations, aiming to improve resource allocation and operational reliability without the complexity of a full-scale cloud orchestration platform.

The Mavericks

# Literature & RFC Review

Existing research on cloud resource provisioning highlights the importance of efficient resource allocation and systematic evaluation of scheduling policies. Calheiros et al. introduced CloudSim, a simulation toolkit for evaluating cloud resource provisioning algorithms, while Beloglazov et al. studied resource-allocation heuristics for improving cloud data-centre efficiency. These studies provide a foundation for designing and evaluating the proposed memory-aware scheduling mechanism in PS-023 [3], [3].

The Cloud Hypervisor API documentation defines the interfaces for managing virtual machines, while Linux cgroup v2 provides mechanisms for organizing processes and controlling resource distribution [4], [5]. Additionally, IETF RFC 9110 specifies HTTP semantics and idempotency principles relevant to handling repeated requests and communication failures. These references inform the project's design for VM lifecycle management, resource control, and reliable failure recovery [6].

# Proposed Solution

## 01 Solution 01

Develop a central controller to manage virtual machines across two KVM-capable Linux hosts using Cloud Hypervisor. It supports VM creation, starting, stopping, listing, and deletion through a unified interface. The controller coordinates requests and maintains VM status information across both hosts.

## 02 Solution 02

Select suitable hosts for new VMs based on available and allocatable memory.
Use resource reservations to prevent concurrent requests from exceeding a host's memory capacity.
This improves resource utilization and reduces VM launch failures caused by insufficient memory.

## 03 Solution 03

Deploy worker agents on both hosts to execute VM operations through the local Cloud Hypervisor API.
Establish authenticated communication between the controller and workers for secure task execution.
Workers report VM status and host health, enabling coordinated and modular management.

## 04 Solution 04

Monitor worker health and reconcile controller records with the actual state of virtual machines.
Handle timeouts and failures by checking VM status before retrying operations to avoid duplicate actions.
Release resource reservations and perform safe cleanup to maintain consistent system state.

The Mavericks

# Optimization Proposed

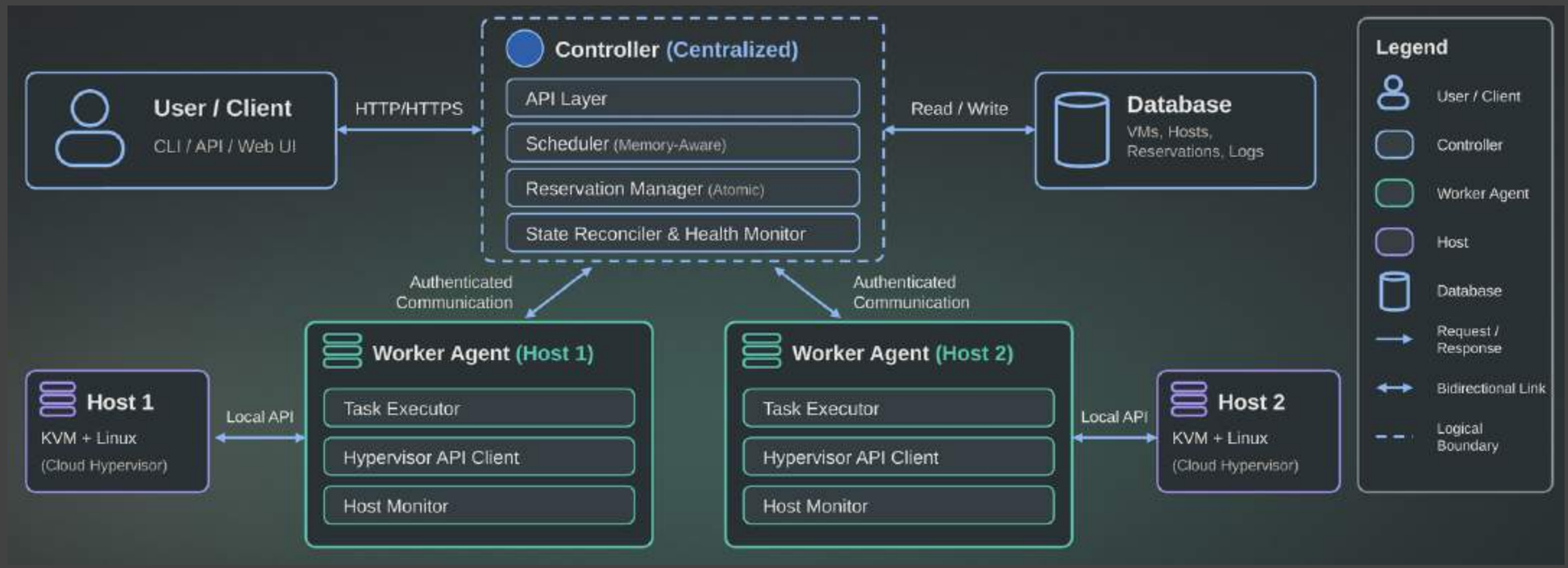
The proposed system improves VM management through memory-aware scheduling, atomic resource reservations, secure communication, and automated failure recovery. The table compares the expected improvements over a basic VM management approach.

These optimizations aim to improve resource utilization, operational reliability, and consistency across both hosts. Effectiveness will be evaluated through functional testing, concurrent VM requests, memory-allocation checks, and simulated worker failures.

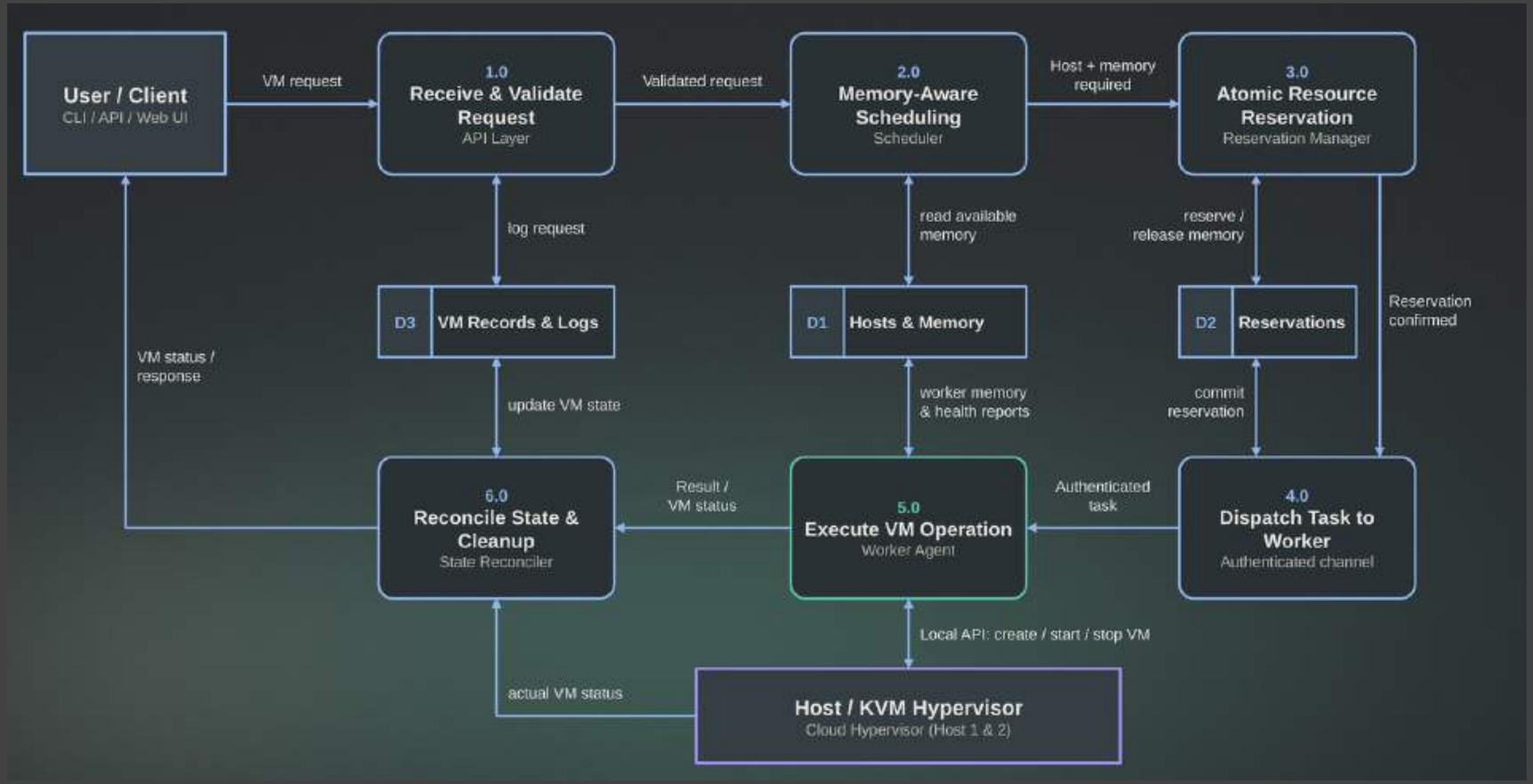
| Aspect | Before Optimization | After Optimization |
|---|---|---|
| VM Scheduling | VM placement based on basic host selection. | Selects hosts based on available and allocatable memory. |
| Resource Allocation | Concurrent requests may cause memory overcommitment. | Atomic reservations prevent resource allocation conflicts. |
| Communication | Limited coordination between management and host operations. | Authenticated communication between controller and worker agents. |
| Failure Handling | Timeouts may cause uncertain VM states or duplicate actions. | State reconciliation checks actual VM status before retries. |
| Resource Cleanup | Failed operations may leave stale reservations. | Safe cleanup releases reservations when appropriate. |
| Monitoring | Limited visibility into worker availability and VM status. | Worker health monitoring and status reporting improve visibility. |

# Solution Architecture & Design

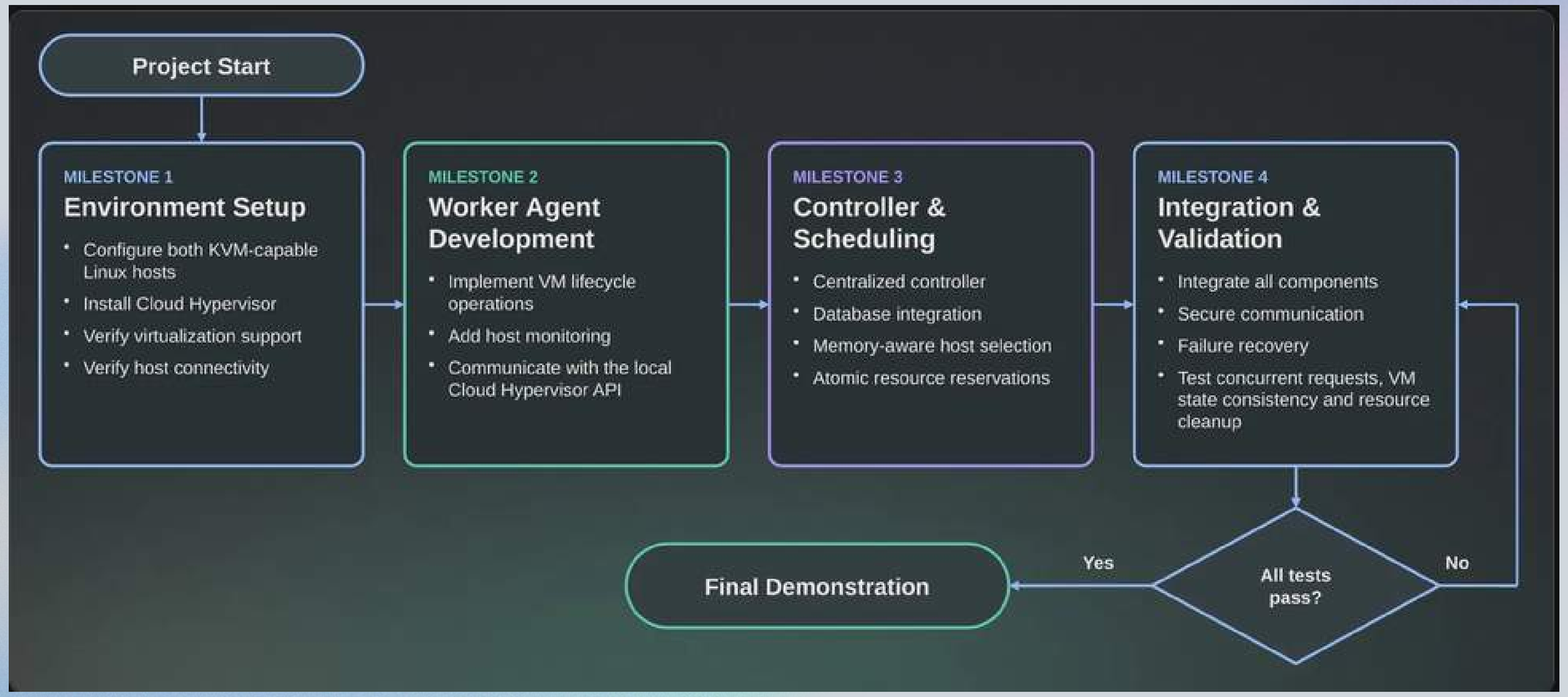
## Visual: Flowchart or diagram showing the optimization process.



# Data Flow Diagram



# Deployment Timeline



The Mavericks

# Bibliography


[1] R. N. Calheiros, R. Ranjan, A. Beloglazov, C. A. F. De Rose, and R. Buyya, “CloudSim: A Toolkit for Modeling and Simulation of Cloud Computing Environments and Evaluation of Resource Provisioning Algorithms,” Software: Practice and Experience, vol. 41, no. 1, pp. 23–50, 2011. https://doi.org/10.1002/spe.995

[2] A. Beloglazov, J. Abawajy, and R. Buyya, “Energy-aware Resource Allocation Heuristics for Efficient Management of Data Centers for Cloud Computing,” Future Generation Computer Systems, vol. 28, no. 5, pp. 755–768, 2012. https://doi.org/10.1016/j.future.2011.04.017

[3] Cloud Hypervisor, “API Documentation.” Available: https://github.com/cloud-hypervisor/cloud-hypervisor/blob/main/docs/api.md

[4] Linux Kernel Documentation, “Control Group v2.” Available: https://docs.kernel.org/admin-guide/cgroup-v2.html

[5] IETF, “RFC 9110: HTTP Semantics,” 2022. Available: https://www.rfc-editor.org/rfc/rfc9110.html

[6] Linux Kernel Documentation, “The Definitive KVM (Kernel-based Virtual Machine) API Documentation.” Available: https://docs.kernel.org/virt/kvm/api.html

# Thank You